

Stack & Linked List

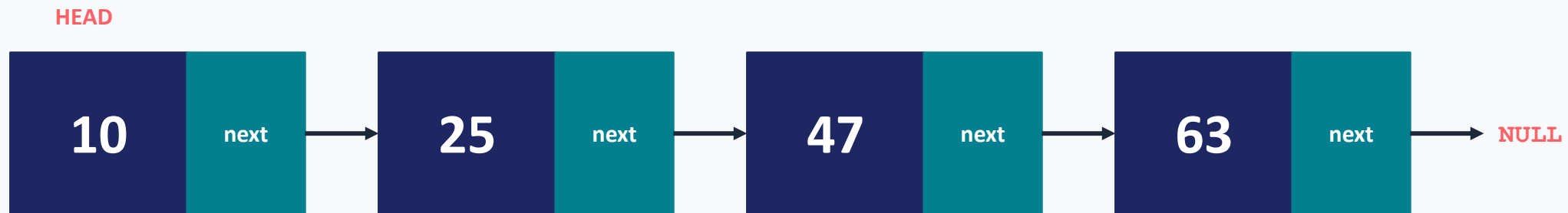
Linked List

A chain of nodes connected by pointers

What is a Linked List?

A linear collection of nodes where each node points to the next

Definition: A Linked List is a dynamic, linear data structure made of self-referential nodes. Each node stores a piece of data and a pointer that links it to the next node in the chain. The first node is called the **head**, and the last node points to **NULL**.



Key Properties

- Dynamic size
- Non-contiguous memory
- Sequential access

Real Uses

- Implementing stacks, queues, graphs
- Music playlists, browser history
- Memory managers and OS process tables

LinkedList class will support seven operations

insertAtHead(v)

Add a new node containing v at the start of the list.

deleteFromHead()

Remove the first node and free its memory; throws if empty.

display()

Print every node from head to NULL in order.

~LinkedList()

Destructor — walks the list and deletes every node.

insertAtTail(v)

Add a new node containing v at the end of the list.

search(v)

Return true if a node with value v exists in the list.

isEmpty()

Return true when head == NULL.

Step 1: Build the Node Class

Every linked list begins with a single node

What does a Node hold?

1. data: the value stored in the node
2. next: a pointer to another Node

Why public?

Node is a helper class that LinkedList will manage. The LinkedList itself will keep its internals private.

```
class Node {  
    public:  
        int data;    // payload  
        Node* next;  // link to next node  
  
    Node(int value) {  
        data = value;  
        next = NULL;  
    }  
};
```

Step 2: Design the LinkedList Class

Decide on the attributes and the public interface

Private attribute:

Node* head: pointer to the first node, or NULL when the list is empty.

Public methods to expose:

- LinkedList() //constructor
- ~LinkedList() //destructor
- insertAtHead(v)
- insertAtTail(v)
- deleteFromHead()
- search(v)
- display()
- isEmpty()

```
class LinkedList {  
private:  
    Node* head;  
  
public:  
    LinkedList();  
    ~LinkedList();  
  
    void insertAtHead(int v);  
    void insertAtTail(int v);  
    void deleteFromHead();  
    bool search(int v);  
    void display();  
    bool isEmpty();  
};
```

Step 3: Constructor & Destructor

Constructor:

Initialize head to NULL so we know the list starts empty.

Destructor:

When the LinkedList object goes out of scope we must free every node we allocated with new, otherwise we leak memory.

Walk-the-list pattern:

Use two pointers: current to point at the node we are deleting, and a temp to remember the next node.

```
LinkedList::LinkedList() {
    head = NULL;
}

LinkedList::~~LinkedList() {
    Node* current = head;
    while (current != NULL) {
        Node* temp = current->next;
        delete current;
        current = temp;
    }
    head = NULL;
}

bool LinkedList::isEmpty() {
    return head == NULL;
}
```

Step 4: insertAtHead() & insertAtTail()

Two ways to grow the list

insertAtHead

1. Create a new Node with new
2. Point its next to current head
3. Move head to the new node

insertAtTail

1. If list is empty, new node becomes head
2. Else walk until current->next is NULL
3. Attach the new node there

```
void LinkedList::insertAtHead(int v) {  
    Node* n = new Node(v);  
    n->next = head;  
    head    = n;  
}
```

```
void LinkedList::insertAtTail(int v) {  
    Node* n = new Node(v);  
    if (head == NULL) {  
        head = n;  
        return;  
    }  
    Node* curr = head;  
    while (curr->next != NULL) {  
        curr = curr->next;  
    }  
    curr->next = n;  
}
```

Step 5: deleteFromHead() — with Exception

What if the user calls delete on an empty list?

Algorithm:

- If head is NULL, throw an exception
- Save head in a temp pointer
- Move head to head->next
- delete the saved temp

```
void LinkedList::deleteFromHead() {  
    if (head == NULL) {  
        throw "List is empty!";  
    }  
    Node* temp = head;  
    head      = head->next;  
    delete temp;  
}
```

Step 6: search() & display()

Read-only traversals of the list

search(v):

Walk from head; if any node's data equals v return true. If we reach NULL, return false.

display():

Same traversal pattern, but print each value separated by an arrow. End with NULL to make the chain visible.

```
bool LinkedList::search(int v) {
    Node* curr = head;
    while (curr != NULL) {
        if (curr->data == v)
            return true;
        curr = curr->next;
    }
    return false;
}

void LinkedList::display() {
    Node* curr = head;
    while (curr != NULL) {
        cout << curr->data << " -> ";
        curr = curr->next;
    }
    cout << "NULL" << endl;
}
```

Step 7: Make It a Template

One LinkedList for int, double, string — anything

Three changes:

1. Add `template <typename T>` above each class.
2. Replace every `int` data type with `T`.
3. Define member functions inside the class — templates and separate `.cpp` files do not mix easily.

Now you can write:

```
LinkedList<int>  nums;  
LinkedList<string> names;
```

```
template <typename T>  
class Node {  
public:  
    T      data;  
    Node*  next;  
    Node(T v) { data = v; next = NULL; }  
};  
  
template <typename T>  
class LinkedList {  
private:  
    Node<T>* head;  
public:  
    LinkedList() { head = NULL; }  
    void insertAtHead(T v) {  
        Node<T>* n = new Node<T>(v);  
        n->next = head;  head = n;  
    }  
    // ... other methods unchanged  
};
```

Step 8: Drive It from main()

Expected output:

30 -> 20 -> 10 -> 40 -> NULL

Found 20: 1

20 -> 10 -> 40 -> NULL

```
int main() {
    LinkedList<int> list;
    list.insertAtHead(10);
    list.insertAtHead(20);
    list.insertAtHead(30);
    list.insertAtTail(40);
    list.display();

    cout << "Found 20: "
         << list.search(20) << endl;

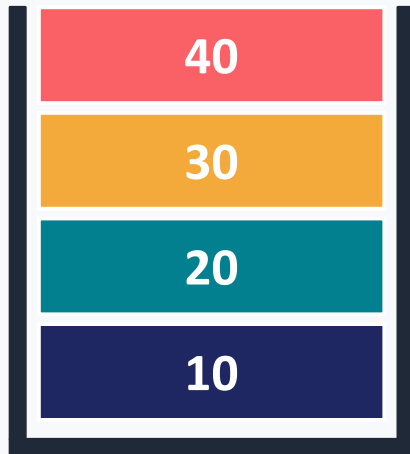
    try {
        list.deleteFromHead();
        list.display();
    }
    catch (const char* e) {
        cout << e;
    }
    return 0;
}
```

Stack

What is a Stack?

A linear LIFO container (Last In, First Out)

Definition: A Stack is a linear data structure where insertions and removals happen at the same end, called the **top**. The element pushed last is the element popped first (LIFO).



Core Operations

push (v)	Insert v at the top.
pop ()	Remove the top element.
top ()	Read the top element without removing it.
isEmpty ()	Returns true when no elements.
isFull ()	Returns true when capacity is reached.
size ()	Number of elements currently stored.

Stack Step 1: Class Design (Array-Based)

Why an array first?

Fixed size, contiguous memory.

Private attributes:

- arr: pointer to a dynamic array on the heap
- capacity: max elements the stack can hold
- topIndex: index of the current top (-1 when empty)

Public methods:

constructor, destructor, push, pop, top, isEmpty, isFull, size

```
class Stack {  
private:  
    int* arr;  
    int  capacity;  
    int  topIndex;  
  
public:  
    Stack(int cap);  
    ~Stack();  
  
    void push(int v);  
    int  pop();  
    int  top();  
    bool isEmpty();  
    bool isFull();  
    int  size();  
};
```

Stack Step 2: Constructor & Destructor

Allocate the array on the heap, free it on exit

Constructor responsibilities:

- Save the requested capacity
- Allocate a new int array of that size
- Set topIndex to -1 to mark empty

Destructor responsibilities:

- delete[] the dynamically allocated array
- Set arr to NULL

Reminder:

- use delete[] for arrays, plain delete for single objects.

```
Stack::Stack(int cap) {
    capacity = cap;
    arr      = new int[cap];
    topIndex = -1;
}

Stack::~~Stack() {
    delete[] arr;
    arr = NULL;
}

bool Stack::isEmpty() {
    return topIndex == -1;
}

bool Stack::isFull() {
    return topIndex == capacity - 1;
}
```

Stack Step 3: push() — with Overflow Exception

Don't write past the end of the array

Algorithm:

1. If isFull(), throw an exception
2. Increment topIndex first
3. Place the new value at arr[topIndex]

Why pre-increment?

topIndex starts at -1 for an empty stack. Incrementing first puts us at index 0, which is the correct slot for the first element.

```
void Stack::push(int v) {  
    if (isFull()) {  
        throw "Stack Overflow!";  
    }  
    topIndex = topIndex + 1;  
    arr[topIndex] = v;  
}
```

Stack Step 4: pop(), top() and size()

Read or remove safely, never on an empty stack.

pop():

1. If empty, throw underflow exception
2. Read arr[topIndex] into a temp
3. Decrement topIndex and return temp

top():

Same emptiness check, but only return. Do not change topIndex.

size():

topIndex + 1 because indices start at zero.

```
int Stack::pop() {
    if (isEmpty()) {
        throw "Stack Underflow!";
    }
    int v = arr[topIndex];
    topIndex = topIndex - 1;
    return v;
}

int Stack::top() {
    if (isEmpty()) {
        throw "Stack is empty!";
    }
    return arr[topIndex];
}

int Stack::size() {
    return topIndex + 1;
}
```

Stack Step 5: Make It a Template

Two changes:

1. `template <typename T>` above the class.
2. Replace the `int` payload type with `T`.

Capacity stays an int. It counts elements, not values.

Now `Stack<T>` can store anything:

```
Stack<int> ints(50);
```

```
Stack<string> words(20);
```

```
Stack<double> vals(100);
```

```
template <typename T>
class Stack {
private:
    T* arr;
    int capacity;
    int topIndex;
public:
    Stack(int cap) {
        capacity = cap;
        arr = new T[cap];
        topIndex = -1;
    }
    ~Stack() { delete[] arr; }

    void push(T v) {
        if (topIndex == capacity - 1)
            throw "Overflow!";
        arr[++topIndex] = v;
    }
};
```

Stack Step 6: Demo in main()

Trace the output yourself:

Top: 30

Popped: 30

Stack Overflow!

```
int main() {  
    Stack<int> s(3);  
    try {  
        s.push(10);  
        s.push(20);  
        s.push(30);  
        cout << "Top: " << s.top();  
        cout << "Popped: " << s.pop();  
        s.push(40);    // ok  
        s.push(50);    // ok  
        s.push(60);    // overflow!  
    }  
    catch (const char* msg) {  
        cout << msg << endl;  
    }  
    return 0;  
}
```

Bonus: Stack Built on a Linked List

Push and pop at the head

Idea:

Reuse the LinkedList we already wrote. The head of the list becomes the top of the stack.

```
template <typename T>
class Stack {
private:
    Node<T>* topNode;
public:
    Stack() { topNode = NULL; }
    void push(T v) {
        Node<T>* n = new Node<T>(v);
        n->next = topNode;
        topNode = n;    }
    T pop() {
        if (topNode == NULL)
            throw "Underflow!";
        Node<T>* t = topNode;
        T val = t->data;
        topNode = topNode->next;
        delete t;
        return val;
    }
};
```